```
00001     // ===== FILE: assets/scripts/core/ads/TikTokAdService.ts =====
00002     import { TikTokRewardAdSlot, TIKTOK_REWARDED_AD_PLACEMENTS } from '../reddit/TikTokAdsConfig';
00003
00004     /**
00005      * TikTokAdService
00006      * 封装 TikTok 激励广告的加载、展示和奖励回调。
00007      * 仅在 TikTok Minis 环境下可用。
00008      */
00009     export class TikTokAdService {
00010         private static _lastCloseRewarded = false;
00011         private static _lastCloseHandled = false;
00012         private static _sessionUnlockSlots: Set<TikTokRewardAdSlot> = new Set();
00013
00014         public static resetSessionUnlocks(): void {
00015             this._sessionUnlockSlots.clear();
00016         }
00017
00018         public static unlockSessionSlot(slot: TikTokRewardAdSlot): void {
00019             this._sessionUnlockSlots.add(slot);
00020         }
00021
00022         public static isSessionSlotUnlocked(slot: TikTokRewardAdSlot): boolean {
00023             return this._sessionUnlockSlots.has(slot);
00024         }
00025
00026         public static wasLastAdCancelled(): boolean {
00027             return this._lastCloseHandled && !this._lastCloseRewarded;
00028         }
00029
00030         private static markCloseResult(rewarded: boolean): void {
00031             this._lastCloseHandled = true;
00032             this._lastCloseRewarded = rewarded;
00033         }
00034
00035         private static markNonCloseFailure(): void {
00036             this._lastCloseHandled = false;
00037             this._lastCloseRewarded = false;
00038         }
00039
00040         private static handleAdError(err: any, slot: TikTokRewardAdSlot): void {
00041             const errorCode = Number(err?.errorCode);
00042             const subErrorCode = Number(err?.subErrorCode);
00043
00044             if (errorCode === 20003 && subErrorCode === 20001) {
00045                 this.showToast(
00046                     'Ad placement inactive. Please enable this placement in TikTok Console.'
00047                 );
00048                 console.warn(
00049                     `[TikTokAdService] Placement inactive for slot=${slot}. Check Monetization > In-App Ads > A
00050                 );
```

```
                return;
            }

            this.showToast('Ad is temporarily unavailable. Please try again later.');
        }

        private static offIfPossible(
            adInstance: any,
            eventName: 'offClose' | 'offError',
            fn: any
        ): void {
            if (typeof adInstance?.[eventName] === 'function') {
                adInstance[eventName](fn);
            }
        }

        private static showWithBestEffort(adInstance: any): Promise<void> {
            if (typeof adInstance?.show !== 'function') {
                return Promise.reject(new Error('Rewarded ad instance has no show()'));
            }

            const runShow = () => {
                const showResult = adInstance.show();
                if (showResult && typeof showResult.then === 'function') {
                    return showResult;
                }
                return Promise.resolve();
            };

            if (typeof adInstance?.load !== 'function') {
                // Some TikTok runtimes only expose show() without load().
                return runShow();
            }

            const loadResult = adInstance.load();
            if (loadResult && typeof loadResult.then === 'function') {
                return loadResult.then(() => runShow());
            }

            return runShow();
        }

        /** 检测当前是否运行在 TikTok 环境 */
        public static isTikTokRuntime(): boolean {
            const g = globalThis as any;
            return g?.__GVR_PLATFORM__ === 'tiktok' || typeof g?.tt !== 'undefined';
        }

        /** 检测 TikTok 广告 SDK 是否可用 */
        public static isAdAvailable(): boolean {
```

```
            if (!this.isTikTokRuntime()) return false;
            const g = globalThis as any;
            return typeof g?.tt?.createRewardedVideoAd === 'function';
        }

        /** 在 TikTok 端显示轻提示 */
        public static showToast(message: string): void {
            const g = globalThis as any;
            try {
                if (typeof g?.tt?.showToast === 'function') {
                    g.tt.showToast({
                        title: message,
                        icon: 'none',
                        duration: 1500,
                    });
                    return;
                }
            } catch (err) {
                console.warn('[TikTokAdService] showToast failed:', err);
            }
            console.warn(`[TikTokAdService] ${message}`);
        }

        /**
         * 展示激励广告并等待结果
         * @param slot 广告位标识
         * @returns true 表示用户看完广告获得奖励，false 表示中途关闭或出错
         */
        public static showRewardedAd(slot: TikTokRewardAdSlot): Promise<boolean> {
            return new Promise<boolean>(resolve => {
                this.markNonCloseFailure();
                if (!this.isAdAvailable()) {
                    console.warn('[TikTokAdService] Ad SDK not available');
                    this.showToast('Ad is temporarily unavailable. Please try again later.');
                    resolve(false);
                    return;
                }

                const placementId = TIKTOK_REWARDED_AD_PLACEMENTS[slot];
                if (!placementId) {
                    console.warn(`[TikTokAdService] No placement ID for slot: ${slot}`);
                    this.showToast('Ad placement is not configured yet.');
                    resolve(false);
                    return;
                }

                const tt = (globalThis as any).tt;

                try {
                    // TikTok rewardedVideoAd 实例只能展示一次；每次展示都新建实例
```

```
                    const adInstance = tt.createRewardedVideoAd({ adUnitId: placementId });

                    let settled = false;
                    const settleOnce = (rewarded: boolean) => {
                        if (settled) return;
                        settled = true;
                        resolve(rewarded);
                    };

                    // 绑定关闭回调（一次性）
                    const onClose = (res: { isEnded: boolean }) => {
                        this.offIfPossible(adInstance, 'offClose', onClose);
                        this.offIfPossible(adInstance, 'offError', onError);
                        const rewarded = res?.isEnded === true;
                        this.markCloseResult(rewarded);
                        console.log(`[TikTokAdService] Ad closed, rewarded=${rewarded}, slot=${slot}`);
                        settleOnce(rewarded);
                    };

                    const onError = (err: any) => {
                        this.offIfPossible(adInstance, 'offClose', onClose);
                        this.offIfPossible(adInstance, 'offError', onError);
                        this.markNonCloseFailure();
                        console.error(`[TikTokAdService] Ad error for slot=${slot}:`, err);
                        this.handleAdError(err, slot);
                        settleOnce(false);
                    };

                    if (typeof adInstance?.onClose === 'function') {
                        adInstance.onClose(onClose);
                    }
                    if (typeof adInstance?.onError === 'function') {
                        adInstance.onError(onError);
                    }

                    // 加载并展示
                    this.showWithBestEffort(adInstance).catch((err: any) => {
                        this.offIfPossible(adInstance, 'offClose', onClose);
                        this.offIfPossible(adInstance, 'offError', onError);
                        this.markNonCloseFailure();
                        console.error(
                            `[TikTokAdService] Failed to load/show ad for slot=${slot}:`,
                            err
                        );
                        this.handleAdError(err, slot);
                        settleOnce(false);
                    });
                } catch (err) {
                    this.markNonCloseFailure();
                    console.error(`[TikTokAdService] Exception for slot=${slot}:`, err);
```

```
                    this.handleAdError(err, slot);
                    resolve(false);
                }
            });
        }
    }

    // ===== FILE: assets/scripts/core/base/BaseComponent.ts =====
    import { Component } from 'cc';

    /**
     * 组件基类
     * 提供通用的生命周期方法和辅助功能
     * 所有游戏组件应继承此类
     */
    export abstract class BaseComponent extends Component {
        /** 组件是否已初始化 */
        protected _initialized: boolean = false;

        /** 组件是否处于激活状态 */
        protected _isActive: boolean = true;

        /**
         * 初始化方法，子类重写此方法进行初始化
         * 在 onLoad 中自动调用
         */
        protected abstract initialize(): void;

        /**
         * 清理方法，子类重写此方法进行资源释放
         * 在 onDestroy 中自动调用
         */
        protected cleanup(): void {
            // 子类重写
        }

        protected onLoad(): void {
            if (!this._initialized) {
                this.initialize();
                this._initialized = true;
            }
        }

        protected onDestroy(): void {
            this.cleanup();
            this._initialized = false;
        }

        protected onEnable(): void {
            this._isActive = true;
```

```
        }

        protected onDisable(): void {
            this._isActive = false;
        }

        /**
         * 安全的延时调用
         * @param callback 回调函数
         * @param delay 延时秒数
         * @returns unschedule 用的 key
         */
        protected delayCall(callback: () => void, delay: number): void {
            this.scheduleOnce(callback, delay);
        }

        /**
         * 取消当前节点的所有调度
         */
        protected cancelAllDelays(): void {
            this.unscheduleAllCallbacks();
        }
    }

    // ===== FILE: assets/scripts/core/base/index.ts =====
    /**
     * 基础导出
     */
    export { Singleton } from './Singleton';
    export { BaseComponent } from './BaseComponent';

    // ===== FILE: assets/scripts/core/base/Singleton.ts =====
    /**
     * 单例基类
     * 所有管理器类应继承此类以实现单例模式
     *
     * @example
     * class AudioManager extends Singleton<AudioManager>() {
     *     public playSound() { ... }
     * }
     * // 使用: AudioManager.instance.playSound()
     */
    export function Singleton<T>() {
        class SingletonClass {
            static _instance: T | null = null;

            public static get instance(): T {
                if (!this._instance) {
                    this._instance = new (this as unknown as new () => T)();
                }
```

```
                return this._instance;
            }

            /**
             * 销毁单例实例
             * 用于场景切换或热更新时重置状态
             */
            public static destroyInstance(): void {
                this._instance = null;
            }

            /**
             * 检查单例是否已创建
             */
            public static hasInstance(): boolean {
                return this._instance !== null;
            }
        }

        return SingletonClass;
    }

    // ===== FILE: assets/scripts/core/bootstrap/ControllerServices.ts =====
    import { ServiceRegistry } from '../managers/ServiceRegistry';
    import { EventManager } from '../managers/EventManager';
    import { GameManager } from '../managers/GameManager';
    import { EffectManager } from '../managers/EffectManager';
    import { WaveService } from '../managers/WaveService';
    import { PoolManager } from '../managers/PoolManager';
    import { HUDManager } from '../../ui/HUDManager';
    import { BuildingManager } from '../../gameplay/buildings/BuildingManager';
    import { WaveManager } from '../../gameplay/wave/WaveManager';
    import { CoinDropManager } from '../../gameplay/economy/CoinDropManager';
    import { BuffCardService } from '../../gameplay/roguelike/BuffCardService';
    import { BuffCardUI } from '../../ui/BuffCardUI';
    import { TowerUpgradeCardService } from '../../gameplay/roguelike/TowerUpgradeCardService';
    import { TowerUpgradeCardUI } from '../../ui/TowerUpgradeCardUI';
    import { HeroWeaponManager } from '../../gameplay/weapons/HeroWeaponManager';
    import { AirdropService } from '../../gameplay/airdrop/AirdropService';
    import { WeaponSelectUI } from '../../ui/WeaponSelectUI';
    import { WeaponBarUI } from '../../ui/WeaponBarUI';
    import { ItemService } from '../../gameplay/items/ItemService';
    import { ChestDropManager } from '../../gameplay/items/ChestDropManager';
    import { ItemCardUI } from '../../ui/ItemCardUI';
    import { ItemBarUI } from '../../ui/ItemBarUI';

    /**
     * ControllerServices
     * GameController 专用服务入口集合
     */
```

```
    export class ControllerServices {
        public get eventManager(): EventManager {
            return ServiceRegistry.get<EventManager>('EventManager') ?? EventManager.instance;
        }

        public get gameManager(): GameManager {
            return ServiceRegistry.get<GameManager>('GameManager') ?? GameManager.instance;
        }

        public get hudManager(): HUDManager {
            return ServiceRegistry.get<HUDManager>('HUDManager') ?? HUDManager.instance;
        }

        public get buildingManager(): BuildingManager {
            return ServiceRegistry.get<BuildingManager>('BuildingManager') ?? BuildingManager.instance;
        }

        public get effectManager(): EffectManager {
            return ServiceRegistry.get<EffectManager>('EffectManager') ?? EffectManager.instance;
        }

        public get waveManager(): WaveManager {
            return ServiceRegistry.get<WaveManager>('WaveManager') ?? WaveManager.instance;
        }

        public get waveService(): WaveService {
            return ServiceRegistry.get<WaveService>('WaveService') ?? WaveService.instance;
        }

        public get poolManager(): PoolManager {
            return ServiceRegistry.get<PoolManager>('PoolManager') ?? PoolManager.instance;
        }

        public get coinDropManager(): CoinDropManager {
            return ServiceRegistry.get<CoinDropManager>('CoinDropManager') ?? CoinDropManager.instance;
        }

        public get buffCardService(): BuffCardService {
            return ServiceRegistry.get<BuffCardService>('BuffCardService') ?? BuffCardService.instance;
        }

        public get buffCardUI(): BuffCardUI {
            return ServiceRegistry.get<BuffCardUI>('BuffCardUI') ?? BuffCardUI.instance;
        }

        public get towerUpgradeCardService(): TowerUpgradeCardService {
            return (
                ServiceRegistry.get<TowerUpgradeCardService>('TowerUpgradeCardService') ??
                TowerUpgradeCardService.instance
            );
```

```
00401         }
00402
00403         public get towerUpgradeCardUI(): TowerUpgradeCardUI {
00404             return (
00405                 ServiceRegistry.get<TowerUpgradeCardUI>('TowerUpgradeCardUI') ??
00406                 TowerUpgradeCardUI.instance
00407             );
00408         }
00409
00410         public get heroWeaponManager(): HeroWeaponManager {
00411             return (
00412                 ServiceRegistry.get<HeroWeaponManager>('HeroWeaponManager') ??
00413                 HeroWeaponManager.instance
00414             );
00415         }
00416
00417         public get airdropService(): AirdropService {
00418             return ServiceRegistry.get<AirdropService>('AirdropService') ?? AirdropService.instance;
00419         }
00420
00421         public get weaponSelectUI(): WeaponSelectUI {
00422             return ServiceRegistry.get<WeaponSelectUI>('WeaponSelectUI') ?? WeaponSelectUI.instance;
00423         }
00424
00425         public get weaponBarUI(): WeaponBarUI {
00426             return ServiceRegistry.get<WeaponBarUI>('WeaponBarUI') ?? WeaponBarUI.instance;
00427         }
00428
00429         public get itemService(): ItemService {
00430             return ServiceRegistry.get<ItemService>('ItemService') ?? ItemService.instance;
00431         }
00432
00433         public get chestDropManager(): ChestDropManager {
00434             return (
00435                 ServiceRegistry.get<ChestDropManager>('ChestDropManager') ?? ChestDropManager.instance
00436             );
00437         }
00438
00439         public get itemCardUI(): ItemCardUI {
00440             return ServiceRegistry.get<ItemCardUI>('ItemCardUI') ?? ItemCardUI.instance;
00441         }
00442
00443         public get itemBarUI(): ItemBarUI {
00444             return ServiceRegistry.get<ItemBarUI>('ItemBarUI') ?? ItemBarUI.instance;
00445         }
00446     }
00447
00448     // ===== FILE: assets/scripts/core/bootstrap/GameplayBootstrap.ts =====
00449     import { Node } from 'cc';
00450     import { MapGenerator } from '../../gameplay/map/MapGenerator';
```

```
import { CombatSystem } from '../../gameplay/combat/CombatSystem';
import { WaveLoop } from '../../gameplay/wave/WaveLoop';

export type GameplayNodes = {
    mapGenerator: MapGenerator;
    combatSystem: CombatSystem;
    waveLoop: WaveLoop;
    waveNode: Node;
};

/**
 * GameplayBootstrap
 * 负责创建与挂载核心玩法系统节点
 */
export class GameplayBootstrap {
    public static build(container: Node, generateMap: boolean = false): GameplayNodes {
        const mapNode = new Node('MapGenerator');
        container.addChild(mapNode);
        const mapGenerator = mapNode.addComponent(MapGenerator);
        if (generateMap) {
            mapGenerator.generateProceduralMap();
        }

        const combatNode = new Node('CombatSystem');
        container.addChild(combatNode);
        const combatSystem = combatNode.addComponent(CombatSystem);

        const waveNode = new Node('WaveLoop');
        container.addChild(waveNode);
        const waveLoop = waveNode.addComponent(WaveLoop);

        return { mapGenerator, combatSystem, waveLoop, waveNode };
    }
}

// ===== FILE: assets/scripts/core/bootstrap/GameResourceLoader.ts =====
import { resources, Prefab, AnimationClip } from 'cc';
import { resolveHeroModelConfig } from '../../gameplay/units/HeroModelConfig';

export type LoadProgressCallback = (loaded: number, total: number) => void;

/**
 * GameResourceLoader
 * Phase 1: 首屏关键资源 — 用 preloadDir 整目录预加载，绕过 GLB 嵌套路径格式问题。
 * Phase 2: 后台静默预加载。
 *
 * preloadDir 会把目录内所有子资源（含 GLB 生成的 Mesh/Prefab/Material）都加载进缓存，
 * 无需知道 GLB 在 Cocos 中实际展开成哪个嵌套路径。
 */
export class GameResourceLoader {
```

```
00501         // Phase 1 directories — fully preloaded before game starts
00502         private static readonly PHASE1_DIRS = [
00503             'ui', // homepage background + any other UI textures — preload first for fast splash→game transitio
00504             'building', // rifle_tower, fencebar, house, spa, barn_3d, gold, radar_3d
00505             'enemies/vehicle', // Tank, Rover, Truck, Turret, RoundRover (wave-1 enemies)
00506             'models/nature', // trees, bushes, rocks
00507             'floor', // all floor textures
00508             'character', // hero GLB + animation clips
00509             'icon', // coin/diamond/weapon/building type icons — needed for HUD & building pads
00510             'textures', // bullet sprites, fog, tower_bullet — needed for combat VFX
00511             'weapons', // weapon models — needed for hero weapon display
00512         ];
00513
00514         // Phase 2 directories — silently preloaded after game begins
00515         private static readonly PHASE2_DIRS = [
00516             'enemies/boss',
00517             'enemies/flying',
00518             'enemies/bullet',
00519             'footman',
00520             'effects',
00521             'shaders',
00522         ];
00523
00524         /** Phase 1: 等 preloadDir 全部完成，每目录算一个进度单位 */
00525         public static loadPhase1(onProgress: LoadProgressCallback): Promise<void> {
00526             const dirs = [...this.PHASE1_DIRS];
00527             const heroClips = this._heroClipPaths();
00528
00529             // hero clips are individual files — treat them as one extra "task"
00530             const total = dirs.length + (heroClips.length > 0 ? 1 : 0);
00531             if (total === 0) {
00532                 onProgress(1, 1);
00533                 return Promise.resolve();
00534             }
00535
00536             return new Promise(resolve => {
00537                 let done = 0;
00538                 onProgress(0, total);
00539
00540                 const tick = () => {
00541                     done++;
00542                     onProgress(done, total);
00543                     if (done >= total) resolve();
00544                 };
00545
00546                 for (const dir of dirs) {
00547                     resources.preloadDir(dir, (_err: Error | null) => tick());
00548                 }
00549
00550                 if (heroClips.length > 0) {
```

```
                    let clipsDone = 0;
                    for (const path of heroClips) {
                        resources.load(path, AnimationClip, () => {
                            clipsDone++;
                            if (clipsDone >= heroClips.length) tick();
                        });
                    }
                }
            });
        }

        /** Phase 2: 后台静默预加载，不阻塞游戏 */
        public static loadPhase2(): void {
            for (const dir of this.PHASE2_DIRS) {
                resources.preloadDir(dir);
            }
            // Also warm up hero prefab in case it wasn't in character/ dir
            const cfg = resolveHeroModelConfig();
            if (cfg.prefabPath) resources.preload(cfg.prefabPath, Prefab);
        }

        // ── internals ──────────────────────────────────────────────────────

        private static _heroClipPaths(): string[] {
            const cfg = resolveHeroModelConfig();
            const paths: string[] = [];
            if (cfg.runClipPath) paths.push(cfg.runClipPath);
            if (cfg.idleClipPath && cfg.idleClipPath !== cfg.runClipPath) {
                paths.push(cfg.idleClipPath);
            }
            return paths;
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/GameStartFlow.ts =====
    import { Node, assetManager } from 'cc';
    import { GameManager } from '../managers/GameManager';
    import { GameConfig } from '../../data/GameConfig';
    import { SpawnBootstrap } from './SpawnBootstrap';
    import { WaveLoop } from '../../gameplay/wave/WaveLoop';
    import { MapGenerator } from '../../gameplay/map/MapGenerator';
    import { ServiceRegistry } from '../managers/ServiceRegistry';
    import { HomePage } from '../../ui/home/HomePage';
    import { HUDManager } from '../../ui/HUDManager';
    import { LoadingScreen } from '../../ui/LoadingScreen';
    import { GameResourceLoader } from './GameResourceLoader';
    import { GameSaveManager } from '../managers/GameSaveManager';
    import type { GameSaveDataV2 } from '../managers/GameSaveManager';
    import { HeroLevelSystem } from '../../gameplay/units/HeroLevelSystem';
    import { HeroWeaponManager } from '../../gameplay/weapons/HeroWeaponManager';
```

```
00601     import { AirdropService } from '../../gameplay/airdrop/AirdropService';
00602     import { BuffCardService } from '../../gameplay/roguelike/BuffCardService';
00603     import { BuildingManager } from '../../gameplay/buildings/BuildingManager';
00604     import { WaveManager } from '../../gameplay/wave/WaveManager';
00605     import { Base } from '../../gameplay/buildings/Base';
00606     import { ItemService } from '../../gameplay/items/ItemService';
00607     import { ShopInventoryStore } from '../diamond/ShopInventoryStore';
00608     import type { ItemId } from '../../gameplay/items/ItemDefs';
00609     import { Hero } from '../../gameplay/units/Hero';
00610     import { EventManager } from '../managers/EventManager';
00611     import { GameEvents } from '../../data/GameEvents';
00612
00613     export type StartContext = {
00614         mapGenerator: MapGenerator | null;
00615         waveLoop: WaveLoop | null;
00616         containers: {
00617             enemy: Node;
00618             soldier: Node;
00619             building: Node;
00620             ui: Node;
00621         };
00622         onSpawned?: (base: Node, hero: Node) => void;
00623         onAudioPreferenceSelected?: (enabled: boolean) => void;
00624         showHomePage?: boolean;
00625         saveData?: GameSaveDataV2 | null;
00626     };
00627
00628     /**
00629      * GameStartFlow
00630      * 负责游戏启动与初始实体生成流程
00631      */
00632     export class GameStartFlow {
00633         private static _tiktokResourcesReadyPromise: Promise<void> | null = null;
00634
00635         public static run(ctx: StartContext): void {
00636             const game = this.gameManager;
00637
00638             // If homepage is requested (default true) and not already playing
00639             if (ctx.showHomePage !== false) {
00640                 this.showHomePage(ctx);
00641                 return;
00642             }
00643
00644             // Direct start (e.g. restart or debug)
00645             this.startGame(ctx);
00646             game.startGame();
00647         }
00648
00649         private static showHomePage(ctx: StartContext) {
00650             HUDManager.instance.setVisible(false);
```

```

            const homeNode = new Node('HomePage');
            ctx.containers.ui.addChild(homeNode);
            const homePage = homeNode.addComponent(HomePage);

            homePage.setOnStartRequested(() => {
                homeNode.destroy();
                const freshCtx = { ...ctx, saveData: null };
                this._showLoadingScreen(freshCtx);
            });

            homePage.setOnContinueRequested(() => {
                const saveData = GameSaveManager.instance.load();
                homeNode.destroy();
                const continueCtx = { ...ctx, saveData };
                this._showLoadingScreen(continueCtx);
            });
        }

        private static _showLoadingScreen(ctx: StartContext) {
            this.ensureTikTokResourcesBundleReady()
                .catch(err => {
                    console.warn(
                        '[GameStartFlow] TikTok resources bundle gate failed, continue with default loader.',
                        err
                    );
                })
                .finally(() => {
                    this._showLoadingScreenAfterBundleReady(ctx);
                });
        }

        private static _showLoadingScreenAfterBundleReady(ctx: StartContext) {
            // onComplete fires AFTER GPU warmup (loading screen still visible during warmup)
            const screen = LoadingScreen.show(ctx.containers.ui, () => {
                GameResourceLoader.loadPhase2();
            });
            const audioChoicePromise: Promise<boolean> = screen
                .waitForAudioChoice()
                .then(enabled => {
                    ctx.onAudioPreferenceSelected?.(enabled);
                    return enabled;
                })
                .catch(err => {
                    console.warn(
                        '[GameStartFlow] Failed to resolve audio preference. Fallback to disabled.',
                        err
                    );
                    ctx.onAudioPreferenceSelected?.(false);
                    return false;
```

```
                });

            const phase1ReadyPromise: Promise<boolean> = GameResourceLoader.loadPhase1(
                (loaded, total) => {
                    if (screen.isValid) screen.setProgress(loaded, total);
                }
            )
                .then(() => true)
                .catch(err => {
                    console.warn(
                        '[GameStartFlow] Phase1 resource load failed, continue to gameplay.',
                        err
                    );
                    return false;
                });

            Promise.all([phase1ReadyPromise, audioChoicePromise]).then(([phase1Ready]) => {
                // Phase 1 in CPU memory. Start game NOW while loading screen covers the scene.
                // The 3D scene renders behind the loading screen, uploading GPU textures.
                this.startGame(ctx);
                this.gameManager.startGame();
                // Restore coins/score AFTER gameManager.startGame() which resets them to INITIAL_COINS.
                if (ctx.saveData) {
                    this.gameManager.setCoins(ctx.saveData.coins);
                    this.gameManager.setScore(ctx.saveData.score);
                }
                if (!screen.isValid) return;
                if (phase1Ready) {
                    // GPU warmup: wait 1.0 s (~60 frames) for textures to upload before revealing scene.
                    screen.scheduleOnce(() => {
                        if (screen.isValid) screen.signalReadyToClose();
                    }, 1.0);
                    return;
                }
                // If phase 1 failed, avoid extra warmup delay and reveal immediately.
                screen.signalReadyToClose();
            });
        }

        private static ensureTikTokResourcesBundleReady(): Promise<void> {
            if (!this.isTikTokRuntime()) return Promise.resolve();
            if (assetManager.getBundle('resources')) return Promise.resolve();
            if (this._tiktokResourcesReadyPromise) return this._tiktokResourcesReadyPromise;

            const loadBundle = () =>
                new Promise<void>((resolve, reject) => {
                    assetManager.loadBundle('resources', err => {
                        if (err) {
                            reject(err);
                            return;
```

```
                        }
                        resolve();
                    });
                });

            this._tiktokResourcesReadyPromise = loadBundle().catch(firstErr => {
                const ttLike = (
                    globalThis as unknown as {
                        tt?: {
                            loadSubpackage?: (options: {
                                name: string;
                                success?: () => void;
                                fail?: (err: unknown) => void;
                            }) => void;
                        };
                    }
                ).tt;

                if (!ttLike?.loadSubpackage) {
                    throw firstErr;
                }

                return new Promise<void>((resolve, reject) => {
                    ttLike.loadSubpackage?.({
                        name: 'resources',
                        success: () => {
                            loadBundle().then(resolve).catch(reject);
                        },
                        fail: reject,
                    });
                });
            });

            return this._tiktokResourcesReadyPromise;
        }

        private static isTikTokRuntime(): boolean {
            const g = globalThis as unknown as { __GVR_PLATFORM__?: unknown; tt?: unknown };
            return g.__GVR_PLATFORM__ === 'tiktok' || typeof g.tt !== 'undefined';
        }

        // Refactored actual start logic (Map generation, Spawning)
        private static startGame(ctx: StartContext) {
            // Show HUD when game starts
            HUDManager.instance.setVisible(true);

            if (ctx.mapGenerator) {
                ctx.mapGenerator.generateProceduralMap();
            }

```

```
            const spawned = SpawnBootstrap.spawn(ctx.containers);
            ctx.onSpawned?.(spawned.base, spawned.hero);

            // Load shop purchases from home screen into ItemService for this run
            const shopItems = ShopInventoryStore.drainItems();
            for (const entry of shopItems) {
                ItemService.instance.addItem(entry.id as ItemId, entry.count);
            }

            const startingWave = ctx.saveData?.waveNumber ?? 1;
            SpawnBootstrap.startWaves(ctx.waveLoop, GameConfig.WAVE.FIRST_WAVE_DELAY, startingWave);

            if (ctx.saveData) {
                this.applyPostStartRestore(ctx.saveData, spawned.base, spawned.hero);
            }
        }

        private static applyPostStartRestore(save: GameSaveDataV2, base: Node, hero: Node): void {
            // NOTE: coins/score are intentionally NOT set here — gameManager.startGame() resets them
            // to INITIAL_COINS after this method runs. They are applied in _showLoadingScreen instead.

            // Suppress the unconditional GAME_START weapon offer — player already has weapons.
            AirdropService.instance.suppressInitialOffer();

            HeroLevelSystem.instance.restoreState(save.heroLevel, save.heroXp);

            if (save.weapons && save.weapons.length > 0) {
                HeroWeaponManager.instance.restoreInventory(save.weapons, save.activeWeaponType);
            }

            if (save.buildings && save.buildings.length > 0) {
                BuildingManager.instance.restoreFromSave(save.buildings);
            }

            // Fast-forward WaveManager boss schedule & lane unlocks to match restored wave.
            if (save.waveNumber > 1) {
                WaveManager.instance.restoreToWave(save.waveNumber);
            }

            const baseComp = base.getComponent(Base);
            if (baseComp) {
                const targetBaseLevel = Math.max(1, Math.floor(save.baseLevel ?? 1));
                if (targetBaseLevel > baseComp.level) {
                    baseComp.restoreToLevel(targetBaseLevel);
                }
                baseComp.syncUpgradePadForCurrentLevel();
                // Replay base-upgrade side effects (hero/barracks scaling) without opening buff-card UI.
                this.eventManager.emit(GameEvents.BASE_UPGRADE_READY, {
                    baseLevel: targetBaseLevel,
                    suppressCardDraw: true,
```

```
                });
            }
            if (baseComp && typeof save.baseHpRatio === 'number') {
                baseComp.currentHp = Math.max(1, Math.floor(save.baseHpRatio * baseComp.maxHp));
                baseComp.refreshHudHp();
            }

            if (save.buffCardIds && save.buffCardIds.length > 0) {
                BuffCardService.instance.restorePickedHistory(save.buffCardIds);
            }

            if (typeof save.nextOfferWave === 'number' && save.nextOfferWave > 1) {
                AirdropService.instance.setNextOfferWave(save.nextOfferWave);
            }

            if (save.items && save.items.length > 0) {
                ItemService.instance.restoreFromSave(save.items);
            }

            const heroComp = hero.getComponent(Hero);
            if (heroComp) {
                heroComp.restoreCoinCount(save.heroCoinCount);
            }

            console.log(
                `[GameStartFlow] Restored save: wave=${save.waveNumber}, baseLv=${Math.max(1, Math.floor(save.b
            );
        }

        private static get gameManager(): GameManager {
            return ServiceRegistry.get<GameManager>('GameManager') ?? GameManager.instance;
        }

        private static get eventManager(): EventManager {
            return ServiceRegistry.get<EventManager>('EventManager') ?? EventManager.instance;
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/ResourcePreloader.ts =====
    import { resources, Texture2D, Prefab, AnimationClip } from 'cc';
    import { resolveHeroModelConfig } from '../../gameplay/units/HeroModelConfig';
    import { GameConfig } from '../../data/GameConfig';

    /**
     * ResourcePreloader
     * 在游戏启动时预加载运行时懒加载资源，避免首波帧率抖动。
     *
     * 调用 preloadAll() 即可触发所有资源的后台加载。
     * 加载结果缓存在各自的工厂/管理器中（通过 resources 引用计数），
     * 后续运行时 load 会命中缓存，不再产生 IO 开销。
```

```
00901      */
00902     export class ResourcePreloader {
00903         private static _started = false;
00904
00905         /** 一次性触发所有关键资源的预加载 */
00906         public static preloadAll(): void {
00907             if (this._started) return;
00908             this._started = true;
00909
00910             this.preloadEnemyPaperDollTextures();
00911             this.preloadSoldierGooseTextures();
00912             this.preloadWeaponVFXTextures();
00913             this.preloadBuildingModelPrefabs();
00914             this.preloadHeroAssets();
00915             this.preloadHeroWeaponVisuals();
00916         }
00917
00918         // === Enemy Paper-Doll 贴图 ===
00919
00920         private static readonly ENEMY_TEXTURE_PATHS = [
00921             'enemies/Robot/Body',
00922             'enemies/Robot/Head',
00923             'enemies/Robot/LeftArm',
00924             'enemies/Robot/RightArm',
00925             'enemies/Robot/LeftLeg',
00926             'enemies/Robot/RightLeg',
00927             'enemies/robovacuum_blue',
00928             'enemies/robovacuum_blue.webp',
00929         ];
00930
00931         private static preloadEnemyPaperDollTextures(): void {
00932             for (const path of this.ENEMY_TEXTURE_PATHS) {
00933                 // 尝试两种路径格式（与 EnemyPaperDollAnimator.loadFrameFromTexture 一致）
00934                 resources.preload(path, Texture2D);
00935                 resources.preload(`${path}/texture`, Texture2D);
00936             }
00937         }
00938
00939         private static readonly SOLDIER_GOOSE_TEXTURE_PATHS = [
00940             'footman/goose/Run',
00941             'footman/goose/Flap',
00942         ];
00943
00944         private static preloadSoldierGooseTextures(): void {
00945             for (const path of this.SOLDIER_GOOSE_TEXTURE_PATHS) {
00946                 resources.preload(path, Texture2D);
00947                 resources.preload(`${path}/texture`, Texture2D);
00948             }
00949         }
00950
```

```
        // === WeaponVFX 贴图 ===

        private static preloadWeaponVFXTextures(): void {
            resources.preload('textures/bullet/texture', Texture2D);
            resources.preload('textures/bullet', Texture2D);
            resources.preload('floor/grass/texture', Texture2D);
            resources.preload('floor/grass', Texture2D);
            resources.preload('floor/grass.webp', Texture2D);
            resources.preload('floor/Dirt_02/texture', Texture2D);
            resources.preload('floor/Dirt_02', Texture2D);
            resources.preload('floor/Dirt_02.webp', Texture2D);
            resources.preload('textures/droplet/texture', Texture2D);
            resources.preload('textures/droplet', Texture2D);
            resources.preload('textures/droplet.webp', Texture2D);
            resources.preload('building/sunflower/texture', Texture2D);
            resources.preload('building/sunflower', Texture2D);
            resources.preload('building/sunflower.webp', Texture2D);
            resources.preload('building/radar_3d', Prefab);
            resources.preload('effects/star_coin', Prefab);
            resources.preload('effects/star_coin/star_coin', Prefab);
        }

        private static readonly BUILDING_MODEL_PREFAB_PATHS = [
            'building/barn_3d',
            'building/barn_3d/barn_3d',
        ];

        private static preloadBuildingModelPrefabs(): void {
            for (const path of this.BUILDING_MODEL_PREFAB_PATHS) {
                resources.preload(path, Prefab);
            }
        }

        // === Hero 模型 + 动画 ===

        private static preloadHeroAssets(): void {
            const config = resolveHeroModelConfig();
            if (!config) return;

            // Prefab
            const prefabPaths = [config.prefabPath, ...config.prefabFallbacks].filter(Boolean);
            for (const p of prefabPaths) {
                resources.preload(p, Prefab);
            }

            // Run clip
            const runPaths = [config.runClipPath, ...config.runClipFallbacks].filter(Boolean);
            for (const p of runPaths) {
                resources.preload(p as string, AnimationClip);
            }
```

```

            // Idle clip
            const idlePaths = [config.idleClipPath, ...config.idleClipFallbacks].filter(Boolean);
            for (const p of idlePaths) {
                resources.preload(p as string, AnimationClip);
            }
        }

        private static preloadHeroWeaponVisuals(): void {
            const visuals = (GameConfig.HERO as unknown as { WEAPON_VISUALS?: unknown }).WEAPON_VISUALS;
            if (!visuals || typeof visuals !== 'object') return;

            const entries = visuals as Record<
                string,
                { prefab?: { path?: string; fallbacks?: string[]; uuid?: string } }
            >;
            for (const key of Object.keys(entries)) {
                const prefabCfg = entries[key]?.prefab;
                const paths = [prefabCfg?.path, ...(prefabCfg?.fallbacks ?? [])].filter(Boolean);
                for (const p of paths) {
                    resources.preload(p as string, Prefab);
                }
            }
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/RuntimeSystemsBootstrap.ts =====
    import { Node } from 'cc';
    import { PlayerInputAdapter } from '../input/PlayerInputAdapter';
    import { BuildingSystemTick } from '../../gameplay/buildings/BuildingSystemTick';

    export type RuntimeSystems = {
        inputAdapter: PlayerInputAdapter;
    };

    /**
     * RuntimeSystemsBootstrap
     * 负责创建运行时系统（输入、建造系统调度等）
     */
    export class RuntimeSystemsBootstrap {
        public static build(container: Node): RuntimeSystems {
            const inputNode = new Node('PlayerInput');
            container.addChild(inputNode);
            const inputAdapter = inputNode.addComponent(PlayerInputAdapter);

            const buildingNode = new Node('BuildingSystem');
            container.addChild(buildingNode);
            buildingNode.addComponent(BuildingSystemTick);

            return { inputAdapter };
```

```
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/SceneGraphBuilder.ts =====
    import { Node } from 'cc';
    import { CoinDropManager } from '../../gameplay/economy/CoinDropManager';
    import { EffectManager } from '../managers/EffectManager';
    import { ServiceRegistry } from '../managers/ServiceRegistry';

    export type SceneGraphNodes = {
        container: Node;
        enemyContainer: Node;
        soldierContainer: Node;
        buildingContainer: Node;
        coinContainer: Node;
        effectContainer: Node;
    };

    /**
     * SceneGraphBuilder
     * 负责创建与组织场景内核心容器节点
     */
    export class SceneGraphBuilder {
        public static build(root: Node): SceneGraphNodes {
            const old = root.getChildByName('GameContainer');
            if (old) {
                old.destroy();
            }

            const container = new Node('GameContainer');
            root.addChild(container);

            const enemyContainer = new Node('Enemies');
            const soldierContainer = new Node('Soldiers');
            const buildingContainer = new Node('Buildings');
            const coinContainer = new Node('Coins');

            container.addChild(enemyContainer);
            container.addChild(soldierContainer);
            container.addChild(buildingContainer);
            container.addChild(coinContainer);

            SceneGraphBuilder.coinDropManager.initialize(coinContainer);

            const effectContainer = new Node('Effects');
            container.addChild(effectContainer);
            SceneGraphBuilder.effectManager.initialize(effectContainer);

            return {
                container,
```

```
                enemyContainer,
                soldierContainer,
                buildingContainer,
                coinContainer,
                effectContainer,
            };
        }

        private static get coinDropManager(): CoinDropManager {
            return ServiceRegistry.get<CoinDropManager>('CoinDropManager') ?? CoinDropManager.instance;
        }

        private static get effectManager(): EffectManager {
            return ServiceRegistry.get<EffectManager>('EffectManager') ?? EffectManager.instance;
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/ServiceRegistrar.ts =====
    import { Node } from 'cc';
    import { ServiceRegistry } from '../managers/ServiceRegistry';
    import { EventManager } from '../managers/EventManager';
    import { GameManager } from '../managers/GameManager';
    import { HUDManager } from '../../ui/HUDManager';
    import { BuildingManager } from '../../gameplay/buildings/BuildingManager';
    import { BuildingRegistry } from '../../gameplay/buildings/BuildingRegistry';
    import { EffectManager } from '../managers/EffectManager';
    import { WaveManager } from '../../gameplay/wave/WaveManager';
    import { WaveService } from '../managers/WaveService';
    import { PoolManager } from '../managers/PoolManager';
    import { CoinDropManager } from '../../gameplay/economy/CoinDropManager';
    import { UnitFactory } from '../../gameplay/units/UnitFactory';
    import { BuffCardService } from '../../gameplay/roguelike/BuffCardService';
    import { BuffCardUI } from '../../ui/BuffCardUI';
    import { TowerUpgradeCardService } from '../../gameplay/roguelike/TowerUpgradeCardService';
    import { TowerUpgradeCardUI } from '../../ui/TowerUpgradeCardUI';
    import { HeroWeaponManager } from '../../gameplay/weapons/HeroWeaponManager';
    import { AirdropService } from '../../gameplay/airdrop/AirdropService';
    import { WeaponSelectUI } from '../../ui/WeaponSelectUI';
    import { WeaponBarUI } from '../../ui/WeaponBarUI';

    /**
     * ServiceRegistrar
     * 统一注册全局服务入口
     */
    export class ServiceRegistrar {
        public static registerCore(): void {
            ServiceRegistry.register('EventManager', EventManager.instance);
            ServiceRegistry.register('GameManager', GameManager.instance);
            ServiceRegistry.register('HUDManager', HUDManager.instance);
            ServiceRegistry.register('BuildingManager', BuildingManager.instance);
```

```
            ServiceRegistry.register('BuildingRegistry', BuildingRegistry.instance);
            ServiceRegistry.register('EffectManager', EffectManager.instance);
            ServiceRegistry.register('CoinDropManager', CoinDropManager.instance);
            ServiceRegistry.register('WaveManager', WaveManager.instance);
            ServiceRegistry.register('WaveRuntime', WaveManager.instance);
            ServiceRegistry.register('WaveService', WaveService.instance);
            ServiceRegistry.register('PoolManager', PoolManager.instance);
            // 注册士兵工厂池（预热 5 个实例，避免首波实例化卡顿）
            PoolManager.instance.registerFactoryPool(
                'soldier_basic',
                () => UnitFactory.createSoldier(null),
                0
            );
            // Fallback spawner when soldier pool is not registered
            ServiceRegistry.register('SoldierSpawner', (parent: Node, x: number, z: number) =>
                UnitFactory.createSoldier(parent, x, z)
            );
            ServiceRegistry.register('BuffCardService', BuffCardService.instance);
            ServiceRegistry.register('BuffCardUI', BuffCardUI.instance);
            ServiceRegistry.register('TowerUpgradeCardService', TowerUpgradeCardService.instance);
            ServiceRegistry.register('TowerUpgradeCardUI', TowerUpgradeCardUI.instance);
            ServiceRegistry.register('HeroWeaponManager', HeroWeaponManager.instance);
            ServiceRegistry.register('AirdropService', AirdropService.instance);
            ServiceRegistry.register('WeaponSelectUI', WeaponSelectUI.instance);
            ServiceRegistry.register('WeaponBarUI', WeaponBarUI.instance);
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/SpawnBootstrap.ts =====
    import { Node } from 'cc';
    import { GameConfig } from '../../data/GameConfig';
    import { BuildingFactory } from '../../gameplay/buildings/BuildingFactory';
    import { UnitFactory } from '../../gameplay/units/UnitFactory';
    import { CameraRig } from '../camera/CameraRig';
    import { BuildingManager } from '../../gameplay/buildings/BuildingManager';
    import { BuildingPadSpawner } from '../../gameplay/buildings/BuildingPadSpawner';
    import { WaveManager } from '../../gameplay/wave/WaveManager';
    import { GameManager } from '../managers/GameManager';
    import { WaveLoop } from '../../gameplay/wave/WaveLoop';
    import { ServiceRegistry } from '../managers/ServiceRegistry';
    import { UIResponsive } from '../../ui/UIResponsive';

    export type SpawnResult = {
        base: Node;
        hero: Node;
    };

    /**
     * SpawnBootstrap
     * 负责初始实体创建与波次/相机/建造点的装配
```

```
     */
    export class SpawnBootstrap {
        public static spawn(containers: { enemy: Node; soldier: Node; building: Node }): SpawnResult {
            const spawnX = GameConfig.MAP.BASE_SPAWN.x;
            const spawnZ = GameConfig.MAP.BASE_SPAWN.z;

            const base = BuildingFactory.createBase(
                containers.building,
                spawnX,
                spawnZ,
                GameConfig.BUILDING.BASE_START_HP
            );

            const hero = UnitFactory.createHero(
                containers.soldier,
                spawnX + GameConfig.MAP.HERO_SPAWN_OFFSET.x,
                spawnZ + GameConfig.MAP.HERO_SPAWN_OFFSET.z
            );

            SpawnBootstrap.waveManager.initialize(containers.enemy, base);
            const cameraPreset = UIResponsive.getGameplayCameraPreset();
            CameraRig.setupFollow(base.scene, hero, cameraPreset.offset, cameraPreset.fov);

            SpawnBootstrap.buildingManager.setHeroNode(hero);
            BuildingPadSpawner.spawnPads(containers.building, SpawnBootstrap.buildingManager);

            return { base, hero };
        }

        public static startWaves(
            waveLoop: WaveLoop | null,
            delaySeconds: number = GameConfig.WAVE.FIRST_WAVE_DELAY,
            startingWave: number = 1
        ): void {
            if (!waveLoop) return;
            waveLoop.initialize(
                SpawnBootstrap.waveManager,
                SpawnBootstrap.gameManager,
                delaySeconds,
                startingWave
            );
            ServiceRegistry.register('WaveLoop', waveLoop);
        }

        private static get gameManager(): GameManager {
            return ServiceRegistry.get<GameManager>('GameManager') ?? GameManager.instance;
        }

        private static get waveManager(): WaveManager {
            return ServiceRegistry.get<WaveManager>('WaveManager') ?? WaveManager.instance;
```

```
        }

        private static get buildingManager(): BuildingManager {
            return ServiceRegistry.get<BuildingManager>('BuildingManager') ?? BuildingManager.instance;
        }
    }

    // ===== FILE: assets/scripts/core/bootstrap/SystemReset.ts =====
    import { GameManager } from '../managers/GameManager';
    import { EventManager } from '../managers/EventManager';
    import { EffectManager } from '../managers/EffectManager';
    import { WaveService } from '../managers/WaveService';
    import { PoolManager } from '../managers/PoolManager';
    import { HUDManager } from '../../ui/HUDManager';
    import { BuffCardUI } from '../../ui/BuffCardUI';
    import { TowerUpgradeCardUI } from '../../ui/TowerUpgradeCardUI';
    import { TowerSelectUI } from '../../ui/TowerSelectUI';
    import { WeaponSelectUI } from '../../ui/WeaponSelectUI';
    import { WeaponBarUI } from '../../ui/WeaponBarUI';
    import { BuildingManager } from '../../gameplay/buildings/BuildingManager';
    import { BuildingRegistry } from '../../gameplay/buildings/BuildingRegistry';
    import { WaveManager } from '../../gameplay/wave/WaveManager';
    import { CoinDropManager } from '../../gameplay/economy/CoinDropManager';
    import { BuffCardService } from '../../gameplay/roguelike/BuffCardService';
    import { TowerUpgradeCardService } from '../../gameplay/roguelike/TowerUpgradeCardService';
    import { HeroWeaponManager } from '../../gameplay/weapons/HeroWeaponManager';
    import { AirdropService } from '../../gameplay/airdrop/AirdropService';
    import { HeroLevelSystem } from '../../gameplay/units/HeroLevelSystem';
    import { ItemService } from '../../gameplay/items/ItemService';
    import { ChestDropManager } from '../../gameplay/items/ChestDropManager';
    import { ItemCardUI } from '../../ui/ItemCardUI';
    import { ItemBarUI } from '../../ui/ItemBarUI';

    /**
     * SystemReset
     * 负责在游戏重启/销毁时的“硬重置”逻辑。
     * 确保所有单例实例被销毁，以便场景重载后能创建全新的状态。
     */
    export class SystemReset {
        private static _sceneHandoffInProgress = false;

        public static markSceneHandoff(): void {
            this._sceneHandoffInProgress = true;
        }

        public static clearSceneHandoff(): void {
            this._sceneHandoffInProgress = false;
        }

        public static consumeSceneHandoff(): boolean {
```

```
            const active = this._sceneHandoffInProgress;
            this._sceneHandoffInProgress = false;
            return active;
        }

        /**
         * 销毁所有单例实例
         *
         * NOTE: 调用此方法后，所有 Manager.instance 将变为 null。
         * 必须保证之后立即进行场景重载或重新初始化，否则会报错。
         */
        public static shutdown(): void {
            console.debug('[SystemReset] Shutting down all singletons...');

            // 核心管理
            GameManager.destroyInstance();
            EventManager.destroyInstance();
            EffectManager.destroyInstance();
            PoolManager.destroyInstance();

            // 游戏玩法
            WaveService.destroyInstance();
            WaveManager.destroyInstance();
            BuildingManager.destroyInstance();
            BuildingRegistry.destroyInstance();
            CoinDropManager.destroyInstance();
            HeroLevelSystem.destroyInstance();
            BuffCardService.destroyInstance();
            TowerUpgradeCardService.destroyInstance();
            HeroWeaponManager.destroyInstance();
            AirdropService.destroyInstance();

            // 道具系统
            ItemService.destroyInstance();
            ChestDropManager.destroyInstance();

            // UI
            HUDManager.destroyInstance();
            BuffCardUI.destroyInstance();
            TowerUpgradeCardUI.destroyInstance();
            TowerSelectUI.destroyInstance();
            WeaponSelectUI.destroyInstance();
            WeaponBarUI.destroyInstance();
            ItemCardUI.destroyInstance();
            ItemBarUI.destroyInstance();

            console.debug('[SystemReset] All singletons destroyed.');
        }
    }

```

```
// ===== FILE: assets/scripts/core/bootstrap/UIBootstrap.ts =====
import { Node, sys } from 'cc';
import { UIFactory } from '../../ui/UIFactory';
import { Joystick } from '../../ui/Joystick';
import { HUDManager } from '../../ui/HUDManager';
import { BuffCardUI } from '../../ui/BuffCardUI';
import { TowerUpgradeCardUI } from '../../ui/TowerUpgradeCardUI';
import { WeaponSelectUI } from '../../ui/WeaponSelectUI';
import { WeaponBarUI } from '../../ui/WeaponBarUI';
import { TowerSelectUI } from '../../ui/TowerSelectUI';
import { UIResponsive } from '../../ui/UIResponsive';
import { ServiceRegistry } from '../managers/ServiceRegistry';

export type UIRefs = {
    canvas: Node;
    joystick: Joystick;
};

/**
 * UIBootstrap
 * 负责创建 UI Canvas、摇杆与 HUD 初始化
 */
export class UIBootstrap {
    public static build(root: Node): UIRefs {
        const canvas = UIFactory.createUICanvas();
        root.addChild(canvas);

        const joystick = UIFactory.createJoystick(canvas);
        if (sys.isBrowser && !UIResponsive.shouldUseTouchControls()) {
            UIFactory.createDesktopMoveHint(canvas);
        }

        UIBootstrap.hudManager.initialize(canvas);
        UIBootstrap.buffCardUI.initialize(canvas);
        UIBootstrap.towerUpgradeCardUI.initialize(canvas);
        UIBootstrap.weaponSelectUI.initialize(canvas);
        UIBootstrap.weaponBarUI.initialize(canvas);
        UIBootstrap.towerSelectUI.initialize(canvas);

        return { canvas, joystick };
    }

    private static get hudManager(): HUDManager {
        return ServiceRegistry.get<HUDManager>('HUDManager') ?? HUDManager.instance;
    }

    private static get buffCardUI(): BuffCardUI {
        return ServiceRegistry.get<BuffCardUI>('BuffCardUI') ?? BuffCardUI.instance;
    }

```

```
        private static get weaponSelectUI(): WeaponSelectUI {
            return ServiceRegistry.get<WeaponSelectUI>('WeaponSelectUI') ?? WeaponSelectUI.instance;
        }

        private static get towerUpgradeCardUI(): TowerUpgradeCardUI {
            return (
                ServiceRegistry.get<TowerUpgradeCardUI>('TowerUpgradeCardUI') ??
                TowerUpgradeCardUI.instance
            );
        }

        private static get towerSelectUI(): TowerSelectUI {
            return ServiceRegistry.get<TowerSelectUI>('TowerSelectUI') ?? TowerSelectUI.instance;
        }

        private static get weaponBarUI(): WeaponBarUI {
            return ServiceRegistry.get<WeaponBarUI>('WeaponBarUI') ?? WeaponBarUI.instance;
        }
    }

    // ===== FILE: assets/scripts/core/camera/CameraFollow.ts =====
    import { _decorator, Component, Node, Vec3 } from 'cc';
    import { ScreenShake } from '../../gameplay/weapons/vfx/ScreenShake';

    const { ccclass, property } = _decorator;

    /**
     * Camera Follow Script
     * Smoothly follows a target node (e.g., Hero)
     */
    @ccclass('CameraFollow')
    export class CameraFollow extends Component {
        @property(Node)
        public target: Node | null = null;

        @property
        public smoothSpeed: number = 0.125;

        @property
        public offset: Vec3 = new Vec3(0, 10, 10); // Default isometric-ish offset

        private _tempVec = new Vec3();

        protected start(): void {
            // If target is set and offset is zero, calculate initial offset relative to it
            if (this.target && this.offset.lengthSqr() === 0) {
                Vec3.subtract(this.offset, this.node.position, this.target.position);
            }

            // Snap immediately on start to ensure correct rotation relative to the offset position
```

```
            this.snap();
        }

        protected lateUpdate(dt: number): void {
            // 驱动屏幕震动（通过修改 offset 实现，零额外开销）
            ScreenShake.update(dt);

            if (!this.target || !this.target.isValid) return;

            // Desired Position
            const desiredPos = this._tempVec;
            Vec3.add(desiredPos, this.target.position, this.offset);

            const currentPos = this.node.position;
            const t = this.smoothSpeed;

            // Manual Lerp for clarity
            const finalX = currentPos.x + (desiredPos.x - currentPos.x) * t;
            const finalY = currentPos.y + (desiredPos.y - currentPos.y) * t;
            const finalZ = currentPos.z + (desiredPos.z - currentPos.z) * t;

            this.node.setPosition(finalX, finalY, finalZ);
        }

        /**
         * Snap camera to target immediately and set rotation
         */
        public snap(): void {
            if (this.target) {
                // Set Position
                const desired = new Vec3();
                Vec3.add(desired, this.target.position, this.offset);
                this.node.setPosition(desired);

                // Set Rotation (Look at target)
                this.node.lookAt(this.target.position);
            }
        }
    }

    // ===== FILE: assets/scripts/core/camera/CameraRig.ts =====
    import { Camera, Node, Vec3 } from 'cc';
    import { CameraFollow } from './CameraFollow';

    /**
     * CameraRig
     * 集中处理相机跟随初始化
     */
    export class CameraRig {
        public static setupFollow(
```